Apache Kafka Installation Guide

Complete installation guide for Apache Kafka on Windows, Linux, and macOS with verification steps.

Table of Contents

- Prerequisites
- Windows Installation
- Linux Installation
- macOS Installation
- Verification Steps
- Common Issues
- Environment Setup

Prerequisites

All Operating Systems

- **Java 8 or higher** (Java 11+ recommended)
- **Minimum 4GB RAM** (8GB+ recommended for production)
- **2GB free disk space** (more for production data)
- **Network access** for downloading packages

Check Java Installation

```
# Check if Java is installed
java -version

# Should show something like:
# openjdk version "11.0.x" or "17.0.x" or higher
```

If Java is not installed: - **Windows**: Download from Oracle or use OpenJDK - **Linux**: `sudo apt install openjdk-11-jdk` (Ubuntu/Debian) or `sudo yum install java-11-openjdk` (RHEL/CentOS) - **macOS**: `brew install openjdk@11` or download from Oracle

---

Windows Installation

Method 1: Manual Installation (Recommended)

1. **Download Kafka** `powershell # Download from https://kafka.apache.org/downloads # Choose "Scala 2.13" version (latest stable) # Example: kafka_2.13-4.1.0.tgz`
2. **Extract Kafka** `powershell # Extract to C:\kafka # You should have: C:\kafka\bin\windows\`
3. **Set Environment Variables** `powershell # Add to System Environment Variables: KAFKA_HOME=C:\kafka PATH=%PATH%;%KAFKA_HOME%\bin\windows`
4. **Create Data Directories** `powershell mkdir C:\kafka\logs\zookeeper mkdir C:\kafka\logs\kafka`

Method 2: Using Chocolatey

```
# Install Chocolatey first (if not installed)
# Then install Kafka
choco install apache-kafka

# Set KAFKA_HOME
$env:KAFKA_HOME = "C:\ProgramData\chocolatey\lib\apache-kafka\tools\kafka_2.13-4.1.0"
```

Method 3: Using WSL (Windows Subsystem for Linux)

```
# Install WSL2 first, then follow Linux instructions
wsl --install
# Follow Linux installation steps inside WSL
```

Linux Installation

Method 1: Package Manager (Ubuntu/Debian)

```
# Update package list
sudo apt update

# Install Java if not present
sudo apt install openjdk-11-jdk

# Add Kafka repository
wget -qO - https://packages.confluent.io/deb/7.4/archive.key | sudo apt-key add
-
sudo add-apt-repository "deb [arch=amd64] https://packages.confluent.io/deb/7.4
stable main"

# Install Kafka
sudo apt update
sudo apt install confluent-kafka
```

Method 2: Manual Installation (All Linux Distributions)

```
# Download Kafka
cd /opt
sudo wget https://downloads.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz

# Extract
sudo tar -xzf kafka_2.13-4.1.0.tgz
sudo mv kafka_2.13-4.1.0 kafka

# Set ownership
sudo chown -R $`USER:`$USER /opt/kafka

# Set environment variables
echo 'export KAFKA_HOME=/opt/kafka' >> ~/.bashrc
echo 'export PATH=$`PATH:`$KAFKA_HOME/bin' >> ~/.bashrc
source ~/.bashrc

# Create data directories
mkdir -p /opt/kafka/logs/zookeeper
mkdir -p /opt/kafka/logs/kafka
```

Method 3: Using Docker (All Linux)

```
# Install Docker if not present
sudo apt install docker.io docker-compose

# Create docker-compose.yml
cat > docker-compose.yml << EOF
version: '3.8'
services:
  kafka:
    image: confluentinc/cp-kafka:latest
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    ports:
      - "9092:9092"
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"
EOF

# Start Kafka
docker-compose up -d
```

---

macOS Installation

Method 1: Homebrew (Recommended)

```
# Install Homebrew if not present
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install Java if needed
brew install openjdk@11

# Install Kafka
brew install kafka

# Kafka will be installed to:
# /opt/homebrew/opt/kafka (Apple Silicon)
# /usr/local/opt/kafka (Intel)

# Set environment variables
echo 'export KAFKA_HOME=/opt/homebrew/opt/kafka/libexec' >> ~/.zshrc
echo 'export PATH=$`PATH:`$KAFKA_HOME/bin' >> ~/.zshrc
source ~/.zshrc
```

Method 2: Manual Installation

```
# Download Kafka
cd /usr/local
sudo curl -O https://downloads.apache.org/kafka/4.1.0/kafka_2.13-4.1.0.tgz

# Extract
sudo tar -xzf kafka_2.13-4.1.0.tgz
sudo mv kafka_2.13-4.1.0 kafka

# Set ownership
sudo chown -R $(whoami):staff /usr/local/kafka

# Set environment variables
echo 'export KAFKA_HOME=/usr/local/kafka' >> ~/.zshrc
echo 'export PATH=$`PATH:`$KAFKA_HOME/bin' >> ~/.zshrc
source ~/.zshrc
```

Method 3: Using MacPorts

```
# Install MacPorts first, then:
sudo port install kafka
```

Verification Steps

1. Check Installation

```
# Check if Kafka binaries are accessible
kafka-topics.sh --version
# Should show: kafka_2.13-4.1.0

# Check Java
java -version
# Should show Java 8+ (11+ recommended)

# Check KAFKA_HOME
echo $KAFKA_HOME
# Should show path to Kafka installation
```

2. Start Kafka (Test Mode)

For KRaft Mode (Kafka 4.0+, No Zookeeper)

```
# Generate cluster ID
kafka-storage.sh random-uuid

# Format storage (first time only)
kafka-storage.sh format -t <cluster-id> -c
$KAFKA_HOME/config/kraft/server.properties

# Start Kafka
kafka-server-start.sh $KAFKA_HOME/config/kraft/server.properties
```

For Traditional Mode (With Zookeeper)

```
# Terminal 1: Start Zookeeper
zookeeper-server-start.sh $KAFKA_HOME/config/zookeeper.properties

# Terminal 2: Start Kafka
kafka-server-start.sh $KAFKA_HOME/config/server.properties
```

3. Test Basic Operations

```
# Create a test topic
kafka-topics.sh --bootstrap-server localhost:9092 --create --topic test-topic -
-partitions 1 --replication-factor 1

# List topics
kafka-topics.sh --bootstrap-server localhost:9092 --list

# Send test message
echo "Hello Kafka!" | kafka-console-producer.sh --bootstrap-server
localhost:9092 --topic test-topic

# Read test message
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-topic
--from-beginning --max-messages 1

# Clean up test topic
kafka-topics.sh --bootstrap-server localhost:9092 --delete --topic test-topic
```

Common Issues

Issue 1: Port Already in Use

```
# Check what's using Kafka ports
lsof -ti:9092,2181

# Kill processes if needed
sudo kill -9 $(lsof -ti:9092,2181)
```

Issue 2: Java Not Found

```
# Windows
set JAVA_HOME=C:\Program Files\Java\jdk-11.0.x

# Linux/macOS
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk
# or
export JAVA_HOME=/usr/local/opt/openjdk@11
```

Issue 3: Permission Denied (Linux/macOS)

```
# Fix ownership
sudo chown -R $`USER:`$USER $KAFKA_HOME

# Fix permissions
chmod +x $KAFKA_HOME/bin/*.sh
```

Issue 4: Memory Issues

```
# Reduce Kafka memory usage (for development)
export KAFKA_HEAP_OPTS="-Xmx512M -Xms512M"
export KAFKA_JVM_PERFORMANCE_OPTS="-server -XX:+UseG1GC -
XX:MaxGCPauseMillis=20"
```

Issue 5: Windows Path Issues

```
# Use full paths if environment variables don't work
C:\kafka\bin\windows\kafka-topics.bat --bootstrap-server localhost:9092 --list
```

Environment Setup

Windows Environment Variables

```
# System Environment Variables
KAFKA_HOME=C:\kafka
JAVA_HOME=C:\Program Files\Java\jdk-11.0.x
PATH=%PATH%;%KAFKA_HOME%\bin\windows;%JAVA_HOME%\bin
```

Linux/macOS Environment Variables

```
# Add to ~/.bashrc (Linux) or ~/.zshrc (macOS)
export KAFKA_HOME=/opt/kafka  # or your installation path
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk  # or your Java path
export PATH=$`PATH:`$KAFKA_HOME/bin:$JAVA_HOME/bin
```

Configuration Files

Basic server.properties (for development)

```
# Kafka Server Configuration
broker.id=1
listeners=PLAINTEXT://localhost:9092
log.dirs=/tmp/kafka-logs
num.network.threads=3
num.io.threads=8
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
num.partitions=1
num.recovery.threads.per.data.dir=1
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
log.retention.hours=168
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000
```

Quick Start Commands

After Installation, Test with These Commands:

```
# 1. Start Kafka (choose one method)
kafka-server-start.sh $KAFKA_HOME/config/kraft/server.properties  # KRaft mode
# OR
zookeeper-server-start.sh $KAFKA_HOME/config/zookeeper.properties &  # Traditional
kafka-server-start.sh $KAFKA_HOME/config/server.properties

# 2. Create topic
kafka-topics.sh --bootstrap-server localhost:9092 --create --topic my-topic --partitions 1 --replication-factor 1

# 3. Test producer (Terminal 1)
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic my-topic

# 4. Test consumer (Terminal 2)
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic my-topic --from-beginning

# 5. Type messages in Terminal 1, see them appear in Terminal 2
```

Production Considerations

Security

- Enable SSL/TLS encryption
- Configure SASL authentication
- Set up proper ACLs (Access Control Lists)

Performance

- Tune JVM heap size based on available memory
- Configure appropriate replication factors
- Set up monitoring with JMX metrics

High Availability

- Run multiple Kafka brokers (cluster)

- Configure proper replication
- Set up monitoring and alerting

Troubleshooting Commands

```
# Check Kafka process
ps aux | grep kafka

# Check Kafka logs
tail -f $KAFKA_HOME/logs/server.log

# List consumer groups
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list

# Check topic details
kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic your-topic

# Reset consumer group offset
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group your-group --reset-offsets --to-earliest --topic your-topic --execute
```

Platform-Specific Notes

Windows

- Use `.bat` files instead of `.sh`: `kafka-topics.bat`
- Use backslashes in paths: `C:\kafka\bin\windows\`
- May need to run as Administrator for some operations

Linux

- Consider using systemd for auto-start: `/etc/systemd/system/kafka.service`
- Check firewall settings: `sudo ufw allow 9092`
- Monitor disk space in `/tmp/kafka-logs`

macOS

- Apple Silicon (M1/M2) may need Rosetta for some Java versions

- Use `brew services` for background services: `brew services start kafka`
- Check Activity Monitor for Java processes

Success Indicators

✅ **Installation Successful When:** - `kafka-topics.sh --version` shows version number - `java -version` shows Java 8+ - Can create and list topics - Producer and consumer can exchange messages - No "command not found" errors

❌ **Installation Issues When:** - "Command not found" errors - "Java not found" errors - Port binding failures (9092, 2181) - Permission denied errors - Out of memory errors

---

Next Steps

After successful installation: 1. **Configure for your project** - Update `server.properties` 2. **Create topics** for your ML pipeline 3. **Test with producers/consumers** 4. **Integrate with your ML system** 5. **Set up monitoring and logging**

Support

Official Documentation

- Apache Kafka Documentation
- Kafka Quickstart

Community Resources

- Kafka Users Mailing List
- Stack Overflow - Apache Kafka
- Confluent Community

Troubleshooting

- Check logs in `$KAFKA_HOME/logs/`
- Verify network connectivity on ports 9092, 2181
- Ensure sufficient memory and disk space
- Validate Java installation and version

Last Updated: September 27, 2025 Compatible with: Kafka 4.1.0, Java 11+